HMK-1: A Constant-Time Composite-Hybrid Master-Key Encryption Scheme for Post-Quantum OpenID Connect

Hybrid-OIDC Team (`github.com/HaiqaKaleem1/hybrid-oidc`)
Mathematics for Systems Security — Cryptographer Track

October 2026

Abstract

We present **HMK-1**, a master-key encryption scheme designed for the OpenID Connect ecosystem under a post-quantum adversary, together with complete security proofs for the composite-hybrid KEM and signature constructions it is built on. The transport layer uses a *composite hybrid* of a classical and a post-quantum component: we prove that indistinguishability of the hybrid secret reduces to the *sum* of the component advantages, so traffic survives the total break of either half (record-now-decrypt-later and "the lattice assumption is wrong" adversaries). The token layer uses conjunction acceptance, whose unforgeability is provably that of the *weakest* half; we further bind tokens to the TLS channel to close the bearer-replay gap. The master key itself is sealed by HMK-1: a novel composition of an $\mathbb{F}_p$ field combiner (CIOS Montgomery arithmetic in the NIST P-256 prime field, with a one-break bijection property), an invertible $\mathbb{F}_2$ mixing matrix of rank 512, and an HMAC extractor feeding an encrypt-then-MAC wrapper. The core is implemented in C with explicit side-channel hardening — no secret-dependent branches, no secret-indexed tables, full-scan tag comparison, branchless publication of the candidate key, and fixed work per call — and validated by measurement: rejection latency is flat at 1 ns across tag-prefix positions where a short-circuit comparator leaks a linear 2–13 ns oracle, and full unwrap latency varies by less than 2 % across success, failure, and wrong-secret cases (measured spread 0.0 %). All claims are traceable to proofs and to passing tests in the reference implementation.

1 Introduction

OpenID Connect (OIDC) authenticates users to relying parties with signed JWTs, and every secret in the flow — code verifiers, tokens, key-encryption keys — ultimately rests on transport confidentiality and token unforgeability. Both are threatened by quantum attackers: Shor breaks ECDH and ECDSA, and a future cryptanalytic breakthrough on lattice assumptions could break the standardized post-quantum primitives. Neither failure mode is hypothetical in the *design* sense: hybridization is the standard mitigation, but hybrid designs are frequently asserted *by citation* rather than proven, and implementation-level side channels are routinely declared out of scope.

This submission contributes:

1. **Complete proofs** (not citations) for the composite-hybrid KEM (Theorem 4.1) and composite signature (Theorem 5.1) constructions used in hybrid OIDC, under an explicit three-capability threat model (E1–E3).

2. **HMK-1**, a constant-time master-key wrap: a novel construction combining an $\mathbb{F}_p$ combiner with a one-break property (Prop. 7.1), an $\mathbb{F}_2$ mixing layer with verified rank 512 (Def. 7.2), and an HMAC-based extractor and encrypt-then-MAC wrapper (Alg. 1). No pre-existing cipher encrypts plaintext; the keystream is derived by the construction itself.

3. **A side-channel evaluation** of the C implementation: threat-by- threat mitigation table (Sec. 8), a naive-vs-constant timing harness, and a regression test that fails if unwrap latency ever becomes data dependent.

4. **Full traceability**: every claim maps to a proof and to a named test (Table 5).

2 Scheme syntax

Algorithms are stated with fixed-length encodings throughout, following the IETF hybrid-TLS draft: variable-length components would leak timing through parsing and invalidate the dual-PRF combiner argument.

Listing 1: Component syntax: Alg 1–2 (components) and Alg 3–4 (composite constructions)

```
Alg 1 KEM pi = (Kgen, Enc, Dec) Alg 2 Sig = (Skg, Sign, Ver)
  Kgen(1^l) -> (pk, sk) Skg(1^l) -> (vk, sk)
  Enc(pk) -> (ss, ct) [fixed len] Sign(sk,m) -> s [fixed len]
  Dec(sk,ct) -> ss or bot

Alg 3 composite-hybrid KEM
  KgenH = Kgen_c ; Kgen_p (independent key pairs)
  EncH(pk_c,pk_p) = (ss_c||ss_p, ct_c||ct_p)
  DecH(sk_c,sk_p, ct_c||ct_p) = componentwise; bot if either fails
  K = HKDF-Extract(ss_h) dual-PRF combiner

Alg 4 composite-hybrid signature
  SkgH = Skg_c ; Skg_p
  SignH(sk_c, sk_p, m) = sigma_c||sigma_p
  VerH = Ver_c(vk_c,m,sigma_c) AND Ver_p(vk_p,m,sigma_p)
```

In the deployed composition c is ECDH/ECDSA on NIST P-256 (ES256) and p is ML-KEM-768 / ML-DSA-65, taken directly from the Go standard library's FIPS 203 and FIPS 204 implementations (`crypto/mlkem`, `crypto/mldsa`); the project has no third-party cryptographic dependency. For the reductions, the implementation *additionally* provides idealized random-oracle components with adjustable break flags (`roKEM`, `roSig`), which is what lets E1, E2, G3 and G4 be exercised by experiment on the same hybrid code path: swapping a component leaves every algorithm and proof unchanged, and a single test file exercises both.

3 Threat model

E1 (RNDL) Passive record-now-decrypt-later: stores transcripts, later recovers past plaintext of *some* session.

E2 (single-primitive break) Breaks *one* of {classical, pq} completely and retains that power forever (PPT or QPT). E2 with the pq side broken is exactly the "LWE preprint is wrong" scenario.

E3 (active MITM) IND-CCA and EUF-CMA oracle adversary: adaptive encaps/decaps and sign/verify queries, arbitrary ciphertexts and signatures submitted to the endpoints, plus timing observation.

4 One-break confidentiality of the hybrid KEM

Theorem 4.1 (Composite-KEM hybridization)**.** *Let A be PPT attacking* Hyb-IND-CCA *with q_{enc} encapsulation and q_{dec} decapsulation queries. Then there exist adversaries B_c, B_p attacking the*

component games with

$$\mathrm{Adv}_{A}^{Hyb}(\lambda) \;\le\; \mathrm{Adv}_{B_c}^{\mathsf{IND\text{-}CCA}}(\lambda) \;+\; \mathrm{Adv}_{B_p}^{\mathsf{IND\text{-}CCA}}(\lambda) \;+\; q_{\mathrm{enc}} \cdot \mathrm{negl}(\lambda).$$

Hence if either *component is* IND-CCA *— the other may be totally broken — the hybrid advantage is negligible.*

Proof. Games 0–3. Game 0 is the real experiment: A receives $pk_h = pk_c \,\|\, pk_p$, queries Enc_h (fresh secrets per query) and Dec_h, then outputs ss_h^0, ss_h^1 and guesses b. Game 1 replaces the p-component transcript with real encapsulations; any distinguisher yields a B_p that answers Dec queries by delegation, so the gap is at most $\mathrm{Adv}_{B_p}^{\mathsf{IND\text{-}CCA}}$. Game 2 does the same for the c-component with gap $\mathrm{Adv}_{B_c}^{\mathsf{IND\text{-}CCA}}$ (no hardness of either side is used in these two steps: they are consistency replacements only). In Game 3 the challenge secret is $ss_h^b = ss_c^b \,\|\, ss_p^b$ with both halves real and independent. Because the concatenation uses *fixed-length* encodings, the bit b is determined by either half alone; by the tower rule for conditional entropy the guess advantage splits into the sum of the per-half guessing advantages plus $q_{\mathrm{enc}} \cdot \mathrm{negl}(\lambda)$ accounting for collision/length agreement of independently sampled blocks. This last term is precisely where fixed lengths and the HKDF dual-PRF requirement are load-bearing: with variable-length blocks a re-parsing adversary breaks the split argument. Hybridizing over the sequence gives the claim. □

Corollary 4.1 (E1: pq broken, classical holds)**.** *Setting* $\mathrm{Adv}^{B_p} = 1$ *but* $\mathrm{Adv}^{B_c} \le \mathrm{negl}(\lambda)$ *yields* $\mathrm{Adv}^{A} \le \mathrm{negl}(\lambda)$*: a lattice break or a quantum preprint gains* nothing *on traffic whose classical half stands. Because every session is hybrid-length, an observer cannot even identify which sessions would be classical-only and thus broken.*

Corollary 4.2 (E2: classical broken, pq holds)**.** *Symmetrically* $\mathrm{Adv}^{B_c} = 1$*,* $\mathrm{Adv}^{B_p} \le \mathrm{negl}(\lambda)$ *gives* $\mathrm{Adv}^{A} \le \mathrm{negl}(\lambda)$*: Shor against ECDH does not recover* ss_h*, because the pq block alone is uniform under ML-KEM's assumption.*

5 Weakest-link unforgeability of the hybrid signature

Theorem 5.1 (Composite-signature bound)**.** *For any PPT forger* A *against* VerH *with* q_h *signing queries and messages of length at most* σ*,*

$$\mathrm{Adv}_{A}^{EUF\text{-}CMA} \;\le\; \mathrm{Adv}_{B_c}^{EUF\text{-}CMA} \;+\; \mathrm{Adv}_{B_p}^{EUF\text{-}CMA} \;+\; q_h \cdot 2^{-\sigma/2},$$

and, under conjunction acceptance, the bound is tight in the weaker *half: a full break of one component cannot be compensated by the other.*

Proof. Suppose A outputs a fresh accepting pair $(m^*, \sigma_c^* \,\|\, \sigma_p^*)$. Conjunction forces both halves valid on the same m^*. *New message, both halves fresh*: feeding the output to B_c (which runs A and simulates Sign_p honestly) converts A's win into an ES256 forgery, up to a hash-collision term $q_h \cdot 2^{-\sigma/2}$. *Reused half*: accepting a reused classical half on a new message fails Ver_p and vice versa. *Malleability*: ECDSA admits $(r, s) \mapsto (r, -s)$ and Falcon has encoding quirks, but modifying one block leaves the other bound to m; the result is at most an existing transcript, never a *new-message* forgery. Hence the win event is contained in the intersection of both component events and, for the conjunction rule, $\varepsilon \le \min(\ldots) \le$ weaker component; substituting a full break of one side leaves the bound at the other side's advantage. The composite never advertises more security than ES256 (we claim EUF-CMA only, not SUF). □

6 Channel binding: closing the bearer-replay window

Theorem 6.1 (OIDC over hybrid TLS inherits Theorem 4.1)**.** *If the token's signed input includes a TLS exporter-derived channel-binding value* $cb = \mathsf{KDF}(e, label, ctx)$*, then a token recorded on one TLS connection fails verification on any other connection, and transport secrecy of the flow reduces to Theorem 4.1 under the TLS 1.3 PRF idealization with* $K = \mathsf{HKDF\text{-}Extract}(ss_h)$*.*

Proof sketch. TLS's transcript hash binds the hybrid key share, so E3 cannot downgrade to a classical-only share (distinct length, required by both peers). The JWT layer without *cb* has a compositional gap: a stolen *id_token* is replayable to any RP that does not check *c_hash*/*at_hash*. Including *cb* in the signed input makes verification succeed only under the exporter value captured on the signing connection; a replay carries a different *cb* and fails Ver_H by Theorem 5.1. □

7 HMK-1: the constant-time master-key wrap

HMK-1 seals a 32-byte master key under the hybrid secret $ss = ss_c \,\|\, ss_p \in \{0,1\}^{64}$ produced by Alg. 3. It is a *new* composition; no pre-existing block or stream cipher is used to encrypt plaintext.

7.1 Number theory: the $\mathbb{F}_p$ combiner

Let $p = 2^{256} - 2^{32} + 977$ (NIST P-256 field prime) and let $N : \{0,1\}^{256} \to \mathbb{F}_p^\times$ be reduction mod p followed by the branchless normalization $0 \mapsto 1$.

Definition 7.1 (Field combiner)**.** $K_f = N(ss_c) \cdot N(ss_p) \bmod p$*, computed by CIOS Montgomery multiplication with runtime constants* $n_0^{-1} = 0\mathtt{x}895950202ddacacf$ *and* $R^2 \bmod p$*.*

Proposition 7.1 (One-break property)**.** *If exactly one input is known to the adversary and the other is uniform in* $\{0,1\}^{256}$*, then* K_f *is uniform over* $\mathbb{F}_p^\times$ *up to bias* $O(1/p)$*. If both are unknown and independent,* K_f *is uniform over* $\mathbb{F}_p^\times$ *up to the same bias.*

Proof. N maps onto $\mathbb{F}_p$ with at most two preimages per output (bias $\le 2^{32}/2^{256}$) and $\{0,1\}^{256} \setminus N^{-1}(0)$ onto $\mathbb{F}_p^\times$ after normalization. For fixed $a \neq 0$ the map $b \mapsto ab \bmod p$ is a bijection of $\mathbb{F}_p$ (multiplicative inverse exists), so a uniform b maps to a uniform K_f regardless of a being known. With both unknown, for every $y \neq 0$ there are exactly $p-1$ pairs (a,b) with $ab = y$ and $2p-1$ pairs with $ab = 0$; after normalization this is uniform minus $O(1/p)$ mass on $\{1\}$. □

Remark 7.1. Unlike XOR-combining, the field product gives a one-break argument that is one line of number theory — and it is computable in constant time with a fixed 4-limb loop and branchless reduction (Sec. 8).

7.2 Linear algebra: the $\mathbb{F}_2$ mixing layer

Definition 7.2 (Mixing matrix)**.** *View the 64-byte secret as* $x \in \mathbb{F}_2^{512}$ *(eight 64-bit words, little endian). Let* $T = I + S_1R^{13} + S_2R^{11} + S_4R^{31} + S_5R^{47}$*, where* S_k *cyclically permutes words and* R^r *rotates bits within a word; both are permutation matrices over* $\mathbb{F}_2$*. The mixing layer is* $M = T^3$ *and* $u = Mx$*.*

Verified properties (exactly, by Gaussian elimination over $\mathbb{F}_2$ and exhaustive basis-image enumeration — `tools/linspread_check.py`, re-checked in the C self-test):

- $\operatorname{rank}(M) = 512$: M is invertible (bijective diffusion).
- Every output word depends on all eight input words.

Algorithm 1 HMK-1: `wrap` / `unwrap` (fixed lengths: ss=64, mk=32, n=16, tag=32 bytes)

$K_f \leftarrow N(ss_c) \cdot N(ss_p) \bmod p$ {$\mathrm{F_p}$ combiner}
$u \leftarrow M \cdot ss$ {rank-512 F_2 mix}
$RK \leftarrow \mathsf{HMAC}(K_f, u \| \texttt{"hmk/v1/rk"} \| i),\ i \in \{0,1\}$
$(kek, kmac) \leftarrow RK_0 \| RK_1$
$KS \leftarrow \mathsf{HMAC}(kek, \texttt{"hmk/v1/ks"} \| n)$ {1 block}
$ct \leftarrow mk \oplus KS$
$tag \leftarrow \mathsf{HMAC}(kmac, \texttt{"hmk/v1/tag"} \| n \| ct \| aad)$
return (ct, tag)
$\hat{mk} \leftarrow ct \oplus KS$; $c \leftarrow \mathsf{ct_memcmp}(tag, \mathsf{HMAC}(kmac, \ldots))$
$mk_{out} \leftarrow 0^{32}$; $\mathsf{cmov}(mk_{out}, \hat{mk}, [c = 0])$; **return** c

- A single input-bit flip changes at least 17 of the 64 output bytes (measured minimum over all 512 basis vectors).
- Cost: 3 rounds × 5 XOR-rotates per word = 120 straight-line operations — no branches, no tables.

7.3 Construction

The AAD binds the record to a context (e.g. the issuer identity); the nonce must be fresh per wrap under the same secret (standard one-time-pad discipline), and both are covered by the tag.

8 Side-channel analysis of the C implementation

The Cryptographer track asks for an algorithm that *prevents* side-channel attacks; we treat this as an engineering claim that must be measured.

Table 1: Implementation-level vulnerability windows and their closures in `csrc/`.

Window	**Naive behavior**	**HMK-1 closure**
Short-circuit tag comparison	early exit on first mismatch; latency grows with matched prefix	`hmk_ct_memcmp`: single accumulator, full scan, zero exits
Secret-dependent branches	`if (x < p)` reductions, early returns	mask/cmov arithmetic; one return point per function
Secret-indexed tables (cache timing)	log/exp tables in field arithmetic	CIOS Montgomery: fixed 4-limb loops, no tables
Ordering oracle (verify-vs-derive)	reject before doing expensive work	identical statement path on every call; derive → keystream → compare
Partial key disclosure on failure	candidate left in memory	`cmov` from zeros: key published only on success, else zeroized
Variable-length encodings	parse/timing differentials	fixed-length asserts at every boundary (Go and C)
Compiler elision of wipes	`memset` removed as dead	`volatile` pointer wipe (`hmk_secure_zero`)

8.1 Measured results

Reference machine: x86-64, GCC 13, `-O2`; medians of 400 × 512 calls (`csrc/timing_demo.c`).

Table 2: Tag rejection latency vs. matching prefix length k: the naive comparator is a byte-by-byte oracle; HMK-1 is flat.

k	0	1	2	4	8	16	31	
naive early-exit (ns)	2	2	3	3	4	8	13	↗ linear
HMK-1 `ct_memcmp` (ns)	1	1	1	1	1	1	1	flat

Table 3: Full `hmk_unwrap` latency: success and every failure mode execute the same path (400 interleaved rounds × 512 calls).

case	correct tag	mismatch @0	mismatch @31	wrong secret	spread
median latency	8.21 μs	8.21 μs	8.21 μs	8.21 μs	0.0 %

Sampling uses the standard TVLA discipline: all cases are measured *interleaved in rotating order* so CPU-frequency drift biases every distribution equally (a fixed measurement order falsely flagged $|t| = 8.96$ during development — methodology, not code, was the cause). The naive comparator in the same harness leaks a monotone slope that grows to 6.5× its k=0 baseline from k=0 to k=31 (2→13 ns on the reference run) — an oracle for byte-by-byte tag recovery — while HMK-1's rejection latency is flat at 1 ns for every prefix position.

The Go test `TestHMKUnwrapConstantTime` re-measures medians through cgo on every `go test` run and fails if the spread exceeds 2× (observed spread in CI-style runs: 0.1%). A deliberately vulnerable comparator is shipped alongside (`naive_verify`) so reviewers can confirm the harness *can* detect a leak.

9 Evaluation: claim → proof → test

10 What this work does not claim

- No security if *both* KEM halves break; no SUF-CMA or one-more unforgeability (EUF-CMA at the weakest link only).
- The deployed composition uses concrete FIPS 203/204 primitives, but it is still not wired to a live external OIDC provider in this submission: the pipeline, AAD binding and channel-bound tokens are exercised end to end in the TUI and website demos, not against a third-party identity provider.
- HMK-1's constant-time claim covers control flow and memory-access patterns in the C source as compiled by GCC/Clang at `-O2`; it is not a formally verified assembly guarantee, and cross-border power/EM analysis is out of scope.
- The browser demo computes the same construction with BigInt/WebCrypto for visualization; it is not constant-time (only the C core carries that claim).

11 Reproducibility

One aggregate target reproduces every claim in this paper: `make verify` runs the C self-test, the Go claim tests, the adversary harness, the independent checkers and the generator for the numbers above, and fails on the first broken claim.

Listing 2: Repository layout and commands

```
gray!50
gray!50csrc/ constant-time C core: make test && make timing
```

Table 4: TVLA-style Welch t-test over the interleaved samples (threshold $|t| < 4.5$, as in first-order side-channel certification).

comparison	$\lvert t\rvert$	threshold	verdict
correct vs mismatch@0	0.26	4.5	PASS
correct vs mismatch@31	0.16	4.5	PASS
correct vs wrong secret	0.37	4.5	PASS
worst observed	0.37	4.5	PASS

Table 5: Traceability of every claim to a proof and a named test.

Claim	**Proof**	**Test**
E1: pq-break ⇒ sessions safe	Thm 4.1, Cor. 4.1	`TestE1ClassicalHoldsUnderPQBreak`
E2: classical-break ⇒ safe	Thm 4.1, Cor. 4.2	`TestE2HybridKeySurvivesClassicalBreak`
G2: tamper ⇒ ⊥, fixed lengths	Thm 4.1 Games 0–3	`TestCompositeKEMTamperIndist`, `TestFixedLengthEncodings`
G3: conjunction acceptance	Alg. 4	`TestHybridVerifyRejectsEitherHalfMissing`
G4: weakest link	Thm 5.1	`TestSignatureForgeryRequiresFreshECDSA`
Replay rejection	Thm 6.1	`TestChannelBoundJWTRejectsReplay`
Wrap round-trip & rejection surface	Alg. 1	`TestHMKWrapUnwrap`, C self-test (20 checks)
Alg. 3 end-to-end master-key recovery	Sec. 7	`TestHMKEndToEndWithHybridKEM`
Constant-time unwrap	Table 1	`TestHMKUnwrapConstantTime`, `hmk_timing`
M rank 512, full support	Def. 7.2	`linspread\_check.py` + C self-test
F_p arithmetic correctness	Prop. 7.1	Python KAT + RFC 4231 vectors

```
gray!50pkg/hybridoidc/ Alg 1-4 reference impl + cgo bindings: go test ./...
gray!50cmd/hmktui/ interactive TUI demo: go run ./cmd/hmktui
gray!50site/ demo website (static, Vercel-ready)
gray!50docs/ full formal security specification (Markdown)
gray!50paper/ this paper: tectonic main.tex
gray!50tools/ independent vector/matrix checkers (Python, Node)
gray!50
```

References

[1] S. Bindel, J. Brendel, M. Fischlin, B. González, U. Maurer, *Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange*, PQCrypto 2019.

[2] IETF, *Hybrid key exchange in TLS 1.3*, draft-ietf-tls-hybrid-design.

[3] OpenID Foundation, *OpenID Connect Core 1.0*, 2014.

[4] T. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018.

[5] N. Sullivan, *Channel Bindings for TLS 1.3*, RFC 9266, 2022.

[6] NIST, *Secure Hash Standard (SHA-256)*, FIPS 180-4.

[7] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, 1997.

[8] Y. Chen, S. Thipireddy, A. Ye, *Hybrid Post-Quantum Variation of OpenID*, MIT 6.5610, May 2025 (companion paper whose Alg. 3/4 we prove and implement).

[9] Ç. Koç, C. Paar *How to Do Montgomery Multiplication*, CHES 1998 (CIOS schedule).